

Algoritmos para economia de energia no escalonamento de workflows em nuvens computacionais

Elaine N. Watanabe¹, Pedro P. V. Campos¹, Kelly R. Braghetto¹, Daniel M. Batista¹

¹Instituto de Matemática e Estatística – Universidade de São Paulo (USP)
Rua do Matão, 1010 – 05508-090 – São Paulo – SP – Brasil

{elainew, pedrovc, kellyrb, batista}@ime.usp.br

Abstract. *Cloud computing is one of the platforms able to provide green computing solutions nowadays. Moreover, it is able to provide a high performance environment for scientific applications. However, if there is dependency between tasks, there may be a low utilization of the cloud. To address the problem, this paper presents new task schedulers for clouds aiming to achieve energy savings. Results of experiments comparing the proposed schedulers with other existing ones show that it is possible to obtain energy savings of up to 22.7%, with no penalty in the makespan. Moreover it is observed that the efficiency of the scheduling is dependent on how tasks are interconnected in the workflow.*

Resumo. *A Computação em Nuvem é uma das plataformas capazes de prover soluções computacionais “verdes” na atualidade. Além disso, ela pode fornecer um ambiente de alto desempenho para aplicações científicas. Entretanto, caso haja dependência entre as tarefas, pode haver uma baixa utilização da nuvem. Para lidar com esse problema, este artigo apresenta novos escalonadores de tarefas para nuvens visando à economia de energia. Resultados de experimentos comparando os escalonadores propostos com outros existentes mostram que é possível obter economias de até 22,7% no consumo de energia, sem haver penalização no makespan. Além disso, comprova-se que a eficiência do escalonamento é dependente do modo como as tarefas estão interligadas no workflow.*

1. Introdução

A Computação Verde (*Green Computing*) tem como objetivo minimizar o impacto ambiental causado pelo uso da tecnologia por meio, por exemplo, do aumento da eficiência energética dos recursos computacionais e da diminuição da emissão de gás carbônico dos *data centers*, responsáveis por 2% do CO_2 emitido no mundo [Beloglazov and Buyya 2010]. Uma utilização ineficiente dos recursos computacionais pode resultar também em gastos extras com a infraestrutura necessária para manter um conjunto de servidores [Barroso and Hölzle 2007]. Estima-se que em 2014, custos com refrigeração e energia elétrica representarão 75% do custo total dos *data centers*, enquanto que o equipamento que essa infraestrutura suporta representará 25% [Belady 2007]. Por esses motivos, tecnologias como a Computação em Nuvem, que visa explorar o máximo dos recursos computacionais minimizando, por exemplo, seu consumo energético, são importantes na atualidade.

Além disso, plataformas de Computação em Nuvem são capazes de fornecer um ambiente de alto desempenho, como é observado no experimento realizado que criou

uma instância de aglomerado com 26.496 núcleos, usando máquinas do tipo *c3.8xlarge* da Amazon EC2, que obteve a posição 64 no rank do TOP500 em novembro de 2013¹. O desempenho dessa instância foi equivalente ao de uma máquina com 484,2 TeraFLOPS. Outra característica importante é a provisão da infraestrutura e serviços sob demanda. Isso possibilita que as aplicações em execução acessem um ambiente elástico, cuja capacidade de processamento pode aumentar ou diminuir automaticamente por meio do uso de “máquinas virtuais” (do inglês *Virtual Machine* – VM). Uma vez que um servidor físico (*host*) pode executar diversas VMs ao mesmo tempo, esse mecanismo permite uma melhor utilização dos recursos disponíveis, resultando em custos menores para o usuário final [Cordeiro et al. 2013]. Assim, a possibilidade de acesso a um ambiente de alto desempenho a um custo reduzido comparado ao de supercomputadores, aliada às características de disponibilização de recursos sob demanda, tornam a Computação em Nuvem uma plataforma vantajosa para a execução de aplicações científicas, em especial, de fluxos de trabalhos (*workflows*) científicos.

Um *workflow científico* consiste na descrição de um experimento científico por meio de um conjunto de tarefas interligadas a serem executadas em computadores. Essas tarefas realizam ações tais como processamento e análise de grandes quantidades de dados, importantes características da ciência moderna. Entretanto, para que um *workflow* seja executado em uma nuvem computacional de forma viável, é essencial que sejam garantidos requisitos como a entrega de um resultado antes de certo tempo ou o controle do orçamento a ser gasto baseado em um limite definido pelo usuário. Nesse contexto, o processo chamado de *escalonamento* tem forte influência no comportamento e desempenho da execução do *workflow* por ser o responsável por determinar o momento do tempo em que cada tarefa será executada e quais recursos serão usados para isso. Esse tipo de processo visa, em geral, otimizar objetivos como o de minimizar o tempo total necessário para a conclusão das tarefas (chamado *makespan*). Além disso, deve garantir, principalmente na execução de *workflows*, que os dados compartilhados entre as tarefas sejam transferidos sempre que necessário. Portanto, um escalonador que não considerar as dependências entre as tarefas pode acabar reduzindo a exploração dos recursos computacionais compartilhados, por exemplo, não aproveitando 100% da potência máxima de um computador, um dos objetivos da Computação em Nuvem, resultando em um consumo energético desnecessário.

Assim, este trabalho tem como objetivo avaliar o escalonamento de *workflows* em nuvens computacionais visando à economia de energia. Para isso, são propostos 3 novos algoritmos, baseados no algoritmo de escalonamento HEFT [Topcuoglu et al. 2002], que avaliam a alocação de recursos com base em seu consumo energético. Experimentos foram realizados com a ferramenta CloudSim-DVFS [Guérout et al. 2013], utilizada para simulação energética de *workflows* em nuvens computacionais, e mostraram economias de energias de até 22,7% sem penalização no *makespan* da aplicação simulada. Tem-se como contribuição neste artigo a proposta de novos mecanismos de escalonamento de *workflows* ciente de energia, assim como um estudo comparativo com outros escalonadores existentes.

O artigo está estruturado da seguinte forma: na Seção 2 os trabalhos relacionados são discutidos e na Seção 3 são descritos os conceitos básicos necessários para o

¹<http://www.top500.org/system/178321>

entendimento do trabalho. A Seção 4 apresenta os novos algoritmos de escalonamento propostos, enquanto que os resultados dos experimentos realizados com esses algoritmos são mostrados na Seção 5. As conclusões e trabalhos futuros são discutidos na Seção 6.

2. Trabalhos relacionados

O uso de ambientes de Computação em Nuvem para execução de workflows é abordado em [Vöckler et al. 2011], que descreve a experiência de uso e análise de testes ao usar o mesmo modelo de execução usado em computação em grade. Entretanto, características como custo financeiro e aquisição de recursos sob demanda diferem as nuvens dos ambientes de grades computacionais e aglomerados, sendo necessárias soluções de escalonamento adaptativas. Em [Bittencourt et al. 2012], são comparados vários algoritmos de escalonamento de workflows e sua aplicabilidade no escalonamento em nuvens. Dentre os mais populares, com mais de mil citações segundo o Google Acadêmico², destaca-se o HEFT (*Heterogeneous Earliest-Finish-Time*) [Topcuoglu et al. 2002], que será descrito na Seção 3 e servirá como base para os novos algoritmos propostos neste artigo.

Em relação à eficiência energética de nuvens computacionais, é destacada em [Beloglazov et al. 2012] a importância de soluções “verdes”. O artigo apresenta um levantamento de projetos de pesquisa que abordam esse problema, destacando desafios de pesquisa em aberto, sob a perspectiva tanto dos provedores de recursos quanto dos consumidores. Propõe, também, princípios de arquitetura e políticas de alocação de recursos que visam ao gerenciamento energeticamente eficiente de nuvens computacionais. Utilizando a ferramenta CloudSim³, um simulador de nuvens computacionais, tais políticas, que empregavam a consolidação dinâmica de VMs, foram avaliadas e os resultados demonstraram grande potencial para a melhoria da eficiência energética para cenários com cargas de trabalho dinâmicas. Entretanto, diferentemente deste trabalho, o artigo não aborda o problema da execução de um fluxo de cargas de trabalho. Inclusive, a ferramenta utilizada (CloudSim) permitia apenas a simulação de uma única carga.

Em [Aksanli et al. 2012], é descrito um estudo comparativo de simuladores de *data center* para avaliar o suporte a simulações energéticas, detalhando um estudo de caso com o simulador GENSim, o único simulador de *data center* capaz de estimar o impacto da alocação de *service jobs* e de *batch jobs* em um único servidor. O objetivo desse estudo foi demonstrar que a predição de energia verde pode melhorar a eficiência energética global de um *data center*. Pelo fato do GENSim não apresentar suporte à simulação de VMs, ele não foi selecionado para a realização dos nossos experimentos.

Em [Kliazovich et al. 2012], é realizada uma comparação entre simuladores de nuvens existentes e é proposto o GreenCloud, que se diferencia do CloudSim por ser capaz de avaliar os componentes da rede, e não apenas os servidores. O GreenCloud é uma extensão do simulador de redes ns-2 e é capaz de capturar, durante a distribuição de carga de trabalho na rede, detalhes tanto do consumo energético de componentes de um *data center* quanto dos padrões de comunicação nos pacotes de dados em configurações realísticas de ambiente. É destacada a importância de se considerar o consumo de energia de enlaces de comunicação, roteadores e outros elementos, visto que eles representam mais de 30% do total de energia consumida pelos *data centers*. Também é demonstrada

²<http://scholar.google.com>

³<http://www.cloudbus.org/cloudsim/>

a aplicabilidade e o impacto do uso de diferentes esquemas de gerenciamento de energia, como o dimensionamento de voltagem ou o desligamento dinâmico, aplicados tanto aos componentes computacionais quanto aos de rede. Entretanto, como o simulador não possui código aberto, não foi escolhido para avaliar os algoritmos propostos neste trabalho.

A fim de habilitar a ferramenta a realizar simulações de execuções de workflows, em [Weiwei and Deelman 2012] foi apresentado o WorkflowSim – uma extensão do simulador CloudSim. O WorkflowSim implementa uma camada para gerenciar a simulação da execução de diversas cargas de trabalho dependentes. Além disso, outras funcionalidades são fornecidas, como o agrupamento de tarefas, que é uma técnica que une pequenas tarefas em grandes “jobs” para reduzir sobrecargas. Os experimentos realizados utilizaram diversos workflows reais descritos na linguagem DAX, usada pelo sistema gerenciador de workflows Pegasus⁴, permitindo que novos algoritmos de escalonamento sejam validados em cenários realísticos. O WorkflowSim possui também mecanismos para simulação de falhas na execução das tarefas e implementa políticas para o tratamento dessas. Entretanto, ele não utiliza o pacote de simulação energética do CloudSim, impossibilitando o uso com simulações que levem em consideração o consumo de energia, como as apresentadas neste artigo.

Este trabalho utiliza o simulador CloudSim-DVFS [Guérout et al. 2013], por ele possibilitar a simulação em nuvens computacionais de workflows descritos no mesmo padrão usado pelo Pegasus, avaliando o tempo de execução e o consumo energético. Além disso, como seu código é livre, foi possível incluir nele os novos algoritmos de escalonamento de tarefas apresentados neste artigo. Mais informações sobre o CloudSim-DVFS serão mostradas nas próximas seções do texto.

3. Conceitos básicos

Nesta seção serão apresentados os conceitos básicos empregados na construção dos algoritmos de escalonamento propostos neste trabalho. Esses algoritmos se baseiam no HEFT. Entretanto, eles têm como objetivo gerar escalonamentos de workflows em nuvens computacionais que resultem em economia de energia quando comparados aos escalonamentos gerados pelo HEFT.

3.1. Modelo energético

Um modelo energético pode ser dividido em duas categorias, conforme descrito em [Rivoire et al. 2008]: modelos completos (*comprehensive models*) e modelos de alto nível chamados de caixa-preta (*black-box*). O primeiro tipo permite obter grande acurácia do consumo energético das CPUs, entretanto, é dependente dos detalhes da microarquitetura dos processadores. O segundo tipo de modelo descreve de forma genérica o consumo energético, não sendo tão preciso. Porém, isso permite ampliar sua portabilidade, não necessitando de detalhes específicos da arquitetura do processador utilizado. Nos experimentos descritos neste trabalho, um modelo de alto nível é utilizado, fornecido pelo simulador utilizado, o CloudSim-DVFS [Guérout et al. 2013]. O modelo utilizado, chamado de modelo linear energético para frequências fixas, é descrito na Equação 1.

$$P_{TOT} = (1 - \alpha)P_{CPUOciosa} + (\alpha)P_{CPUCargaMaxima} \quad (1)$$

⁴<http://pegasus.isi.edu>

onde α é o uso de CPU e $P_{CPUOciosas}$ e $P_{CPUCargaMaxima}$ são os valores da energia consumida pela CPU com 0% e 100% de utilização, respectivamente.

3.2. Representação de workflows como grafos orientados

Diversos workflows científicos de diferentes áreas da ciência podem ser descritos como Grafos Dirigidos Acíclicos (*Directed Acyclic Graph – DAG*) [Deelman et al. 2009]. Para isso, o DAG representa o conjunto de tarefas da aplicação como vértices no grafo e demonstra a dependência entre elas por meio de arcos, indicando a precedência entre as tarefas. Tanto os vértices quanto os arcos podem ser rotulados com valores, que em geral indicam, para os vértices, o custo de computação e, para os arcos, o custo de comunicação, como é representado na Figura 1. Essa representação é a base para o escalonamento, que deve ser feito de forma a respeitar as restrições de precedência, ao mesmo tempo que busca otimizar uma função objetivo.

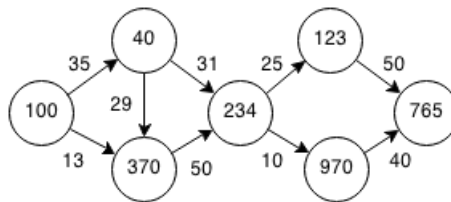


Figura 1. Grafo dirigido acíclico. Na representação de workflows, os vértices representam as tarefas, enquanto os arcos representam dependência e precedência entre as tarefas. Normalmente os vértices são rotulados com o custo computacional; e os arcos, com o custo de comunicação.

As dependências podem ser representadas quanto ao fluxo de dados, indicando que uma tarefa não pode ser executada antes que seus dados de entrada estejam disponíveis, ou quanto ao fluxo de controle, que define que uma tarefa só pode ser executada quando as tarefas das quais ela depende tenham sido concluídas. O caminho mais longo de um vértice inicial até um vértice final é chamado *caminho crítico*, em que o tempo mínimo de execução é obtido a partir da duração das tarefas nesse caminho. As demais tarefas, que se encontram fora desse caminho, são chamadas tarefas não-críticas.

3.3. Algoritmo de escalonamento HEFT

O algoritmo *Heterogeneous Earliest Finish Time* (HEFT) é utilizado para o escalonamento de aplicações modeladas como DAGs para um número limitado de computadores heterogêneos. Devido à sua simplicidade e baixa complexidade, é considerado um dos algoritmos mais populares para o escalonamento estático de workflows [Bittencourt and Madeira 2012].

O HEFT é um algoritmo baseado em listas de tarefas. Ele faz uso de dados históricos para prever o desempenho das tarefas nos recursos computacionais e, a partir do desempenho previsto, realizar o escalonamento. Ele associa tarefas a prioridades atribuindo um peso a cada vértice e arco no DAG, ordenando-as em uma lista, de modo que as tarefas de maior prioridade estão no início da lista e são escalonadas primeiro. O peso para cada tarefa consiste no custo médio de execução da tarefa em cada recurso computacional, enquanto o peso de cada arco compõe a média dos custos de comunicação entre as tarefas, ou seja, custos de transferência de dados. A atribuição das prioridades

utiliza a Equação 2 para ordenar as tarefas, calculando de maneira recursiva o *rank* para cada tarefa, navegando pelo DAG de baixo para cima (da tarefa final para a tarefa inicial).

$$rank(t_i) = \overline{w}_i + \max_{t_j \in suc(t_i)} (\overline{c}_{i,j} + rank(t_j)) \quad (2)$$

onde $\overline{w}_i$ é a média do tempo de execução da tarefa t_i sobre todos os recursos, $suc(t_i)$ são todas as tarefas sucessoras imediatas de t_i , $\overline{c}_{i,j}$ é a média dos custos de comunicação entre t_i e t_j . Basicamente, *rank* é o comprimento do caminho crítico de uma tarefa t_i até a tarefa final do fluxo, incluindo o custo de computação da tarefa t_i . No caso de mais de um caminho possível até a saída, é escolhido o de peso máximo. Após essa etapa de construção da lista (fase de priorização), a tarefa pronta com peso mais alto é selecionada e associada ao recurso que tem o menor tempo de execução esperado para concluir essa tarefa (fase de seleção). O processo se repete até que todas as tarefas estejam escalonadas.

3.4. Simulador CloudSim-DVFS

O CloudSim-DVFS [Guérout et al. 2013] é uma extensão do simulador CloudSim, disponibilizada para download em outubro de 2013. Essa ferramenta permite simulações energéticas tanto de uma carga de trabalho quanto de workflows, além de fornecer uma nova versão da simulação da técnica de dimensionamento dinâmico de voltagem e frequência (*Dynamic voltage and frequency scaling* – DVFS). Essa técnica permite mudar dinamicamente a voltagem e a frequência de um processador de acordo com a carga de trabalho, resultando em um consumo de energia menor.

Para a execução de uma única carga de trabalho no simulador, há os mesmos modos descritos no kernel do Linux para definir o desempenho do processador: (i) *Performance*, que utiliza a CPU na frequência máxima; (ii) *PowerSave*, que utiliza a CPU na menor frequência; (iii) *User-Space*, que permite que o usuário defina uma frequência para a CPU dentre uma lista disponível; (iv) *Conservative* que utiliza um limite superior e um inferior para decidir mudanças na frequência da CPU – caso a carga de trabalho esteja acima do limite superior, a frequência é aumentada; no caso contrário, ela é reduzida – ; e (v) *OnDemand*, que utiliza um único limite – caso a carga de trabalho da CPU o ultrapasse, a frequência da CPU é aumentada para a máxima; caso a carga esteja abaixo desse limite por um determinado período de tempo, ela é reduzida.

A ferramenta utiliza para simulação do DVFS uma lista de tipos de VMs que podem ser instanciadas baseadas nos valores fornecidos pelo modelo energético adotado. Por padrão, é utilizado o modelo energético com os valores de consumo de energia dos *hosts* do Grid’5000 Reims⁵, descritos na Tabela 1. Além disso, é alocada uma tarefa por VM.

Na simulação de workflows, o módulo permite o uso do escalonador DVFS em três modos: (i) *Performance*, onde a VM sempre estará configurada com a frequência máxima disponível; (ii) *OnDemand*, que permite que o sistema mude dinamicamente a frequência da CPU, atribuindo a frequência máxima quando a CPU está sendo utilizada e a mínima quando a CPU encontra-se no estado ocioso; e (iii) *Optimal*, que considera a existência de caminhos críticos e não-críticos no workflow em execução, atribuindo, assim, VMs com frequências máximas para tarefas no caminho crítico e, para tarefas nos

⁵<https://www.grid5000.fr/mediawiki/index.php/Reims:Hardware>

Tabela 1. Frequências disponíveis no Grid'5000 REIMS

Frequências disponíveis (GHz)	CPU	0.8	1.0	1.2	1.5	1.7
Consumo(W)	Ociosa	140	146	153	159	167
	Carga Máxima	228	238	249	260	272

demais caminhos, são atribuídas VMs com frequências ótimas, ou seja, frequências cujo menor consumo energético foi obtido considerando a razão entre o tempo restante para conclusão da tarefa (*slack-time*) e o tempo de execução necessário para a tarefa.

4. Algoritmos propostos

O escalonamento em nuvens computacionais pode ser dividido na alocação de tarefas em máquinas virtuais (VMs) e na alocação de máquinas virtuais em servidores físicos. Este trabalho foca no primeiro caso, propondo escalonar as tarefas de maneira estática com o uso de duas versões estendidas do HEFT, visando à economia de energia. Como não se tem previamente um número fixo de VMs, ambas as abordagens propõem a alocação de VMs dinamicamente baseando-se em uma lista com tipos de VMs fornecidas, com diferentes opções de processadores, mesmo método utilizado em [Guérout et al. 2013].

O primeiro algoritmo proposto baseia-se na variante Lookahead do HEFT [Bittencourt et al. 2010], em que o escalonamento de uma tarefa em uma VM leva em consideração também informações sobre o impacto dessa decisão nas tarefas dependentes da que está sendo alocada. No algoritmo proposto neste trabalho, chamado de PowerHEFTLookahead, a seleção da VM foi modificada, considerando a estimativa de consumo energético ao invés do tempo de conclusão da tarefa.

O segundo algoritmo proposto, chamado de HEFT-DynamicAllocationVM (HEFT-DAVM), modifica a seleção de VMs do HEFT baseando-se no tempo mínimo de início de uma tarefa de acordo com suas tarefas antecessoras e compara esse tempo mínimo com o menor tempo de início dessa mesma tarefa em cada VM disponível em uma dada lista, alocando sempre uma nova VM caso uma sobrecarga de tarefas seja detectada.

Além disso, a partir de uma abordagem descrita em [Weiwei and Deelman 2012], propomos o uso de um algoritmo simples para agrupamento de tarefas em VMs, objetivando uma melhor utilização dos servidores físicos, uma vez que o modelo adotado pelo simulador aloca apenas uma tarefa por VM. E, a fim de comparação do consumo energético com essa abordagem, a técnica de agrupamento é aplicada tanto ao algoritmo proposto em [Guérout et al. 2013] quanto ao algoritmo proposto HEFT-DAVM.

Desse modo, temos os seguintes algoritmos propostos:

- **PowerHEFTLookahead (Algoritmo 1):** inicialmente, há uma lista V de VMs, com apenas uma VM alocada – a mais rápida disponível na lista O de tipos de VMs que podem ser instanciadas. As tarefas são ordenadas conforme os critérios definidos pelo *rank*, descrito na Seção 3.3, e são alocadas em ordem decrescente, como é observado nas linhas 1-5 do algoritmo. Nas linhas 9-11, para cada tarefa t , o algoritmo EscalonarPowerHEFT é chamado. Este algoritmo, que não é apresentado por falta de espaço, força a alocação da tarefa t em uma VM disponível e, em seguida, as tarefas sucessoras de t são escalonadas seguindo os critérios do

HEFT original (apresentado na Seção 3.3), retornando o consumo energético parcial dessa alocação, com base no tempo de processamento da VM escolhida e no modelo linear descrito Seção 3.1. Assim, o Algoritmo 1 memoriza a VM que maximiza a economia de energia. Nas linhas 12-19 do Algoritmo 1, é verificado se seria mais vantajoso energeticamente alocar mais uma VM para o processamento, verificando cada opção de VM disponível. A linha 20 escalona a tarefa na VM que minimiza a energia consumida e a lista de VMs V e o $rank$ são atualizados se necessário.

- **HEFT-DAVM (Algoritmo 2):** a alocação de apenas uma VM e a criação de uma lista ordenada de tarefas conforme o $rank$ definido pela Equação 2 é idêntica ao Algoritmo 1, diferenciando-se na seleção e alocação de novas VMs. Para cada tarefa a ser escalonada, na linha 7 é calculado o tempo mínimo de início de uma tarefa t , baseado no tempo de execução de suas tarefas antecessoras. Em seguida, nas linhas 8-11, o algoritmo HEFT é aplicado, selecionando a VM que tem o tempo mais cedo de conclusão e definindo o tempo de início efetivo de t nessa VM, que depende das tarefas já alocadas nessa máquina. Sempre que for detectada uma sobrecarga nas máquinas, indicada pelo tempo mínimo de início e o tempo de início efetivo, é alocada uma nova VM, a mais lenta disponível (segundo a Tabela 1, a de 0.8 GHz). Espera-se que essa escolha garanta uma economia de energia sem prejudicar o *makespan*. Além disso, visando a redução de outras sobrecargas, como em casos em que uma VM do tipo mais lenta executa um caminho crítico, o que resulta em um *makespan* maior e, conseqüentemente, em um consumo de energia desnecessário dependendo do número de VMs que ficaram aguardando as tarefas desta serem concluídas, todas as VMs que foram selecionadas para executar mais de uma tarefa são “promovidas” a VMs do tipo mais rápida nas linhas 15-18, visando obter uma eficiência energética melhor. Esse modelo assume também que as VMs em que o escalonamento é executado pertencem a uma nuvem privada homogênea.
- **Task Clustering:** uma vez que todas as tarefas foram alocadas, uma heurística é aplicada. Tarefas que serão processadas em VMs do tipo mais lenta são agrupadas em uma nova VM, do tipo mais rápida. Essa técnica visa minimizar o número de VMs utilizadas sem impactar negativamente o *makespan*.

5. Análise de desempenho

A seguir serão descritos a metodologia adotada para as simulações, os resultados obtidos e a análise dos resultados.

5.1. Metodologia

Para avaliar os escalonadores propostos, foram utilizados três workflows reais descritos na linguagem de modelagem DAX (o Sipht, o CyberShake e o Montage) com estruturas distintas e com diversos números de tarefas. Esses workflows são fornecidos pelo sistema Pegasus⁶ e, por retratarem diferentes características importantes para workflows, já foram usados como estudo de caso em vários trabalhos relacionados à área de gerenciamento de workflows científicos. Suas execuções foram simuladas no CloudSim-DVFS.

⁶http://pegasus.isi.edu/workflow_gallery/

Algoritmo 1: PowerHEFTLookahead()

```

1: //V é o conjunto de VMs usadas ao escalonar
2: //VmMaisRápida no modelo descrito na Tabela 1 é a máquina de 1.7 GHz
3:  $V = \{VmMaisRápida\}$ 
4: O = os tipos de VMs que podem ser instanciadas
5: Ordene o conjunto de tarefas de modo decrescente segundo o critério rank
6: enquanto há tarefas não escalonadas faça
7:    $t =$  a tarefa não escalonada de maior rank
8:   // Vamos tentar escalonar t em uma VM existente
9:   para cada  $v$  em  $V$  faça
10:      $energia = EscalonarPowerHEFT(t, v)$  //Retorna consumo energético do HEFT tradicional
11:   fim para
12:   para cada  $o$  em  $O$  faça
13:      $n =$  Simular criação de uma VM do tipo  $o$ ;
14:      $V = V \cup \{n\}$ 
15:     Atualize os valores de rank
16:      $t =$  a tarefa não escalonada de maior rank
17:      $energia = EscalonarPowerHEFT(t, n)$ 
18:     Retorne  $V$  e o escalonamento para os valores do começo do laço
19:   fim para
20:   Escalone  $t$  na VM que minimiza a energia consumida
21:   Atualize  $V$  e rank caso necessário
22: fim enquanto

```

Algoritmo 2: HEFT-DynamicAllocationVM()

```

1: Aloque uma VM do tipo {VMMaisRápida}
2: Defina a média dos custos computacionais das tarefas e dos custos de comunicação
3: Calcule rank para todas as tarefas
4: Ordene as tarefas em uma lista de escalonamento utilizando
   uma ordem decrescente de valores de rank
5: enquanto há tarefas não escalonadas na lista faça
6:   Selecione a primeira tarefa,  $t$  da lista de escalonamento
7:   Calcule o tempo mínimo para execução da tarefa  $t$ 
   com base nas tarefas das quais  $t$  dependa
8:   para cada VM  $m$  no conjunto de VM ( $m \in P$ ) faça
9:     Calcule o tempo de início da tarefa e conclusão da tarefa  $t$ 
     considerando que ela execute em  $m$ 
10:  fim para
11:  Defina o tempo mais cedo de conclusão da tarefa  $t$ 
   e o tempo de início na VM em que esse tempo foi obtido
12:  se o tempo de início for maior que o tempo mínimo para execução de  $t$  então
13:    Aloque uma nova VM do tipo {VMMaisLenta}
14:  senão
15:    se a VM escolhida não é do tipo {VMMaisRápida} então
16:      Aloque uma nova VM do tipo {VMMaisRápida}
17:      Migre todas as tarefas da VM antiga
18:      Defina a tarefa  $t$  para executar na nova VM
19:    senão
20:      Defina a tarefa  $t$  para ser executar na VM que
      minimiza o tempo de conclusão dessa tarefa
21:    fim se
22:  fim se
23: fim enquanto

```

Os escalonadores avaliados foram: (i) modo Performance (PERF.); (ii) DVFS modo Optimal (OPT.); (iii) DVFS modo OnDemand (ONDEM.); (iv) HEFT; (v) PowerHEFTLookahead (POWERHEFT); (vi) HEFT-DAVM (DAVM); (vii) HEFT-DAVM-Task Clustering (DAVM-TC); (viii) DVFS modo Optimal com Task Clustering (OPT-TC). O número de VMs utilizadas para o HEFT foi gerado aleatoriamente, escolhendo-se um dos cinco tipos de VM adotados para as simulações, limitando o número de VMs ao número de tarefas de cada workflow. Os valores apresentados para esse caso são a média e o desvio padrão baseados em trinta simulações. As métricas utilizadas são o *makespan* (o tempo de término da última tarefa de cada workflow) em segundos e o consumo energético total em Watt-hora, fornecidos pelo simulador.

5.1.1. Workflows utilizados

A seguir são descritos os workflows utilizados nos experimentos:

- **Sipht:** utilizado em [Guérout et al. 2013] para validar o CloudSim-DVFS, esse workflow tem como objetivo automatizar a codificação de genes, sendo um projeto de bioinformática de Harvard, cujo exemplo de uma instância é apresentado na Figura 2a.
- **CyberShake:** esse workflow é utilizado pelo Centro de Terremoto do Sul da Califórnia para caracterizar os riscos de terremotos em uma região. Sua estrutura é representada na Figura 2b.
- **Montage:** é uma aplicação criada pela NASA/IPAC, cuja estrutura é ilustrada na Figura 2c, que une diversas imagens de entrada para criar um mosaico personalizado do céu.

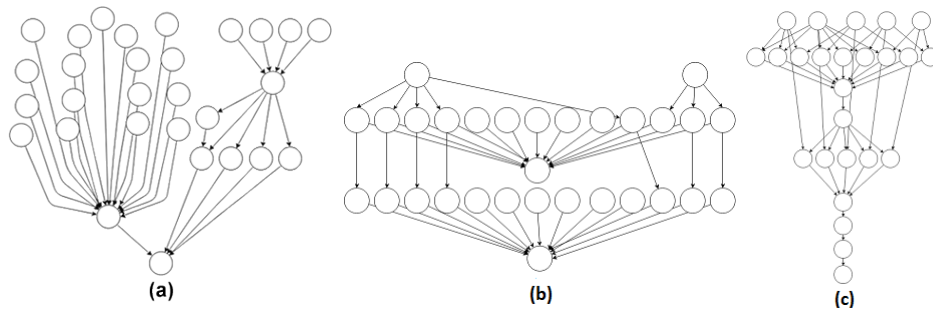


Figura 2. Workflows usados nos experimentos: (a) Sipht, (b) Cybershake e (c) Montage.

5.1.2. Configurações do CloudSim-DVFS

Todas as configurações são definidas no arquivo `simulation.properties`, disponível no simulador. As configurações utilizadas nos experimentos foram as seguintes:

- **Data center:** foram utilizadas as mesmas configurações apresentadas em [Guérout et al. 2013]. Ou seja, o *data center* foi configurado com 500 servidores com o mesmo modelo energético do Grid'5000 REIMS. Esse modelo está incorporado ao pacote power do CloudSim na classe

`PowerModelSpecPower_REIMS`. Para os experimentos, foi considerado que cada servidor possuía 8 núcleos com 1000 MIPS cada.

- **Tipos de VMs utilizadas:** foram utilizados cinco tipos de VMs, com requisitos de 941.2 MIPS, 1176.4 MIPS, 1411.8 MIPS e 2000 MIPS, descritas na classe `VMOffersSimple` do pacote `workflow` do simulador. Cada VM instanciada possuía um único núcleo.
- **Escalonadores:** os escalonadores são alternados no arquivo `simulation.properties`, adicionando-se a referência para a variável `scheduling.policy`. O escalonador HEFT foi implementado no simulador a fim de comparar seu desempenho com as adaptações propostas. A técnica de agrupamento de tarefas (*Task Clustering*) foi aplicada ao HEFT-DAVM e ao OPT estendendo cada classe.
- **Workflows:** a configuração do workflow utilizado é definida na propriedade `dag.file`. Para os experimentos realizados, foram adotados os modelos: (i) `Sipht_30.xml` (30 tarefas), `Sipht_60.xml` (60 tarefas), `Sipht_100.xml` (100 tarefas), (ii) `Montage_25.xml` (25 tarefas), `Montage_50.xml` (50 tarefas), `Montage_100.xml` (100 tarefas) e (iii) `CyberShake_30.xml` (30 tarefas), `CyberShake_50.xml` (50 tarefas), `CyberShake_100.xml` (100 tarefas).

Todos os códigos, dados e documentação dos experimentos realizados estão disponíveis em https://github.com/pedropaulovc/CloudSim_DVFS. O objetivo de manter essas informações públicas é permitir a reprodução dos experimentos futuramente por outros pesquisadores.

5.2. Resultados e discussões

A Figura 3 ilustra os resultados obtidos para o DAG `Sipht`. Adotando a mesma comparação feita em [Guérout et al. 2013] utilizando o modo `Performance` como referência, é possível observar que todos os escalonadores obtiveram valores menores de consumo energético para um DAG com 30 tarefas. Ao realizar o mesmo experimento com mais tarefas, é possível observar que a abordagem de agrupamento de tarefas, DAVM-TC e OPT-TC, mostrou-se vantajosa, superando a eficiência energética e o *makespan* obtidos pelas versões sem essa técnica (DAVM e OPT). É importante observar que apesar do modo ONDEM ter sido o mais eficiente energeticamente e em relação ao *makespan*, os algoritmos propostos (POWERHEFT, DAVM e DAVM-TC) apresentaram reduções no consumo de energia em todos os cenários quando comparados com o algoritmo HEFT original que foi usado como base para as suas construções. Além disso, em termos de *makespan*, com exceção do algoritmo POWERHEFT, não houve perdas significativas. Em média os *makespans* dos algoritmos DAVM e DAVM-TC ficaram apenas 2,85% acima dos valores de *makespan* obtidos com o HEFT original. Os bons resultados obtidos com os métodos que utilizam DVFS apontam que uma direção interessante seria integrar os mecanismos de DVFS aos nossos algoritmos propostos. Os resultados obtidos com a simulação do DAG `CyberShake` foram similares àqueles encontrados com o DAG `Sipht`.

Os experimentos com o DAG `Montage`, um DAG caracterizado por ter caminhos mais longos entre a tarefa de entrada e a tarefa de saída, diferiram dos resultados anteriores. O escalonador HEFT e as nossas propostas baseadas nele (com exceção do POWERHEFT) mostraram-se mais eficientes, até mesmo que o modo ONDEM, tanto energeticamente quanto em relação ao *makespan*. As médias dos ganhos de economia

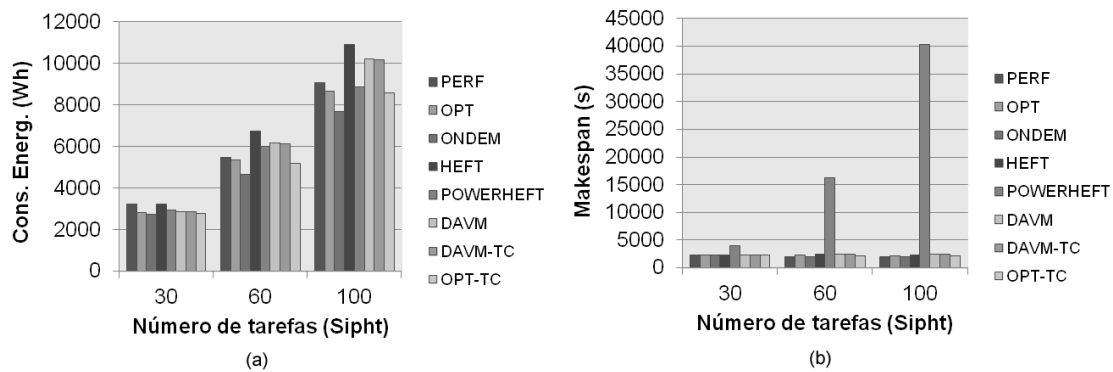


Figura 3. Comparativo do (a) consumo energético em Watt-hora e (b) *makespan* em segundos do DAG Sipht utilizando diferentes políticas de escalonamento de tarefas.

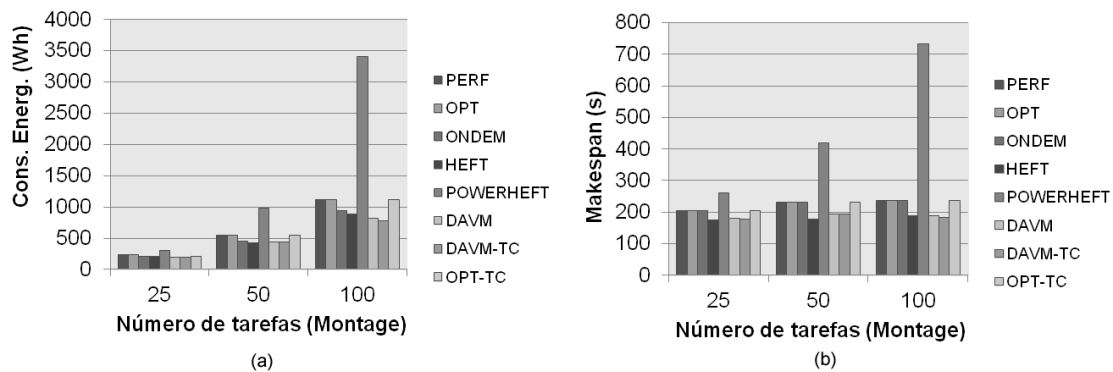


Figura 4. Comparativo do (a) consumo energético em Watt-hora e do (b) *makespan* em segundos do DAG Montage utilizando diferentes políticas de escalonamento de tarefas.

energética obtidos, superiores ao modo PERF, foram: 15,5% (ONDEM), 18,9% (HEFT), 21,1% (DAVM), 22,7% (DAVM-TC) e 5% (OPT-TC). Em relação ao *makespan*, apenas o HEFT, o DAVM e o DAVM-TC apresentaram reduções em relação ao PERF: 19,3%, 15,9% e 17,2%, respectivamente. A técnica de agrupamento de tarefas (TC) não obteve desempenho significativo nos resultados mostrados pela Figura 4.

5.3. Discussão e análises

Do ponto de vista energético, o algoritmo POWERHEFT obteve um ganho médio superior ao modo PERF apenas para o workflow Sipht, contudo, foi inferior aos ganhos médios obtidos pelos algoritmos DVFS nos modos Opt e OnDemand. A técnica de agrupamento de tarefas mostrou-se interessante tanto em relação ao consumo energético quanto ao *makespan*, obtendo, na maioria dos casos, um ganho superior ao do algoritmo original, como no caso do DAVM-TC em relação ao DAVM e do OPT-TC em relação ao OPT. O modo ONDEM obteve melhor desempenho na maioria dos casos, como é descrito por [Guérout et al. 2013]. Entretanto, foi possível observar que nem sempre ele é a melhor escolha, uma vez que na execução do workflow Montage, os melhores ganhos médios obtidos em relação à economia de energia foram dos algoritmos DAVM e DAVM-TC, propostos neste trabalho, apresentando ganhos consideráveis em relação também ao

makespan. Neste mesmo caso, o modo OPT não obteve um ganho significativo uma vez que o Montage não possui caminhos não-críticos, não permitindo a otimização proposta pelo algoritmo.

Assim, como pode ser observado pelos resultados das simulações, os algoritmos propostos conseguem na maioria dos casos melhorar o consumo de energia em relação ao escalonador HEFT original, sem perdas significativas em termos de *makespan*, e em alguns casos conseguem apresentar os melhores ganhos quando comparados com todos os outros escalonadores avaliados. É importante observar que o tamanho do caminho entre as tarefas de entrada e de saída dos DAGs tem influência no desempenho dos escalonadores.

6. Considerações finais e trabalhos futuros

Nuvens computacionais exibem um grande potencial para execução de workflows. Contudo, se o escalonamento de suas tarefas nesse ambiente não considerar o consumo energético, pode resultar na baixa utilização dos recursos disponíveis e, consequentemente, em um maior impacto ambiental, uma vez que os *data centers* já são responsáveis por 2% das emissões de CO_2 no mundo.

Neste artigo, foram propostos três algoritmos para escalonamento de workflows baseados no HEFT, visando a eficiência energética sem impacto significativo no *makespan* das aplicações. O uso de um simulador permitiu avaliar os escalonadores com workflows reais de maneira controlável e facilmente reproduzível. Os resultados obtidos a partir das simulações com diversos workflows, comparados a escalonadores existentes na ferramenta utilizada, permitiram avaliar o desempenho dessas novas abordagens, conseguindo-se obter a redução do consumo energético na maioria dos casos. Foi possível também avaliar os ganhos obtidos com o agrupamento de tarefas, que se mostrou vantajosa tanto em relação ao consumo energético quanto ao tempo de execução (*makespan*). Além disso, os experimentos comprovaram que a eficiência dos escalonadores para workflows depende da estrutura de interligações das tarefas.

Como trabalho futuro, podem ser incorporados: (i) a simulação desses mesmos experimentos com diferentes servidores e modelos energéticos; (ii) a proposta de novas abordagens com o HEFT; (iii) a validação dos resultados em ambientes reais; (iv) a integração dos modos de operação do DVFS com os nossos algoritmos propostos; (v) a implementação no simulador CloudSim-DVFS de simulações de falhas e de diferentes cargas de trabalho por servidor, como descritos no WorkflowSim; (vi) a avaliação de escalonadores dinâmicos ao invés de estáticos; e (vii) a incorporação de estimativa de consumo energético de outros componentes dos servidores, como no uso de servidores para armazenamento de dados.

Agradecimentos

Os autores agradecem à CAPES e à FAPESP (processo número: 2011/24114-0) pelo apoio financeiro, ao Rodrigo N. Calheiros e ao Tom Guérout por disponibilizarem o código do simulador CloudSim-DVFS, ao Weiwei Chen pelo suporte ao WorkflowSim e ao Daniel A. Cordeiro pelos esclarecimentos sobre algoritmos escalonadores de tarefas.

Referências

Aksanli, B., Venkatesh, J., and Rosing, T. (2012). Using datacenter simulation to evaluate green energy integration. *Computer*, 45(9):56–64.

- Barroso, L. and Hölzle, U. (2007). The case for energy-proportional computing. *Computer*, 40(12):33–37.
- Belady, C. (2007). In the data center, power and cooling costs more than the IT equipment it supports. <http://www.electronics-cooling.com/2007/02/in-the-data-center-power-and-cooling-costs-more-than-the-it-equipment-it-supports/> [Online; acessado em 07 de outubro de 2013].
- Beloglazov, A., Abawajy, J., and Buyya, R. (2012). Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. *Future Generation Computer Systems*, 28(5):755–768.
- Beloglazov, A. and Buyya, R. (2010). Energy efficient resource management in virtualized cloud data centers. In *10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing*, pages 826–831.
- Bittencourt, L. F. and Madeira, E. (2012). Escalonamento de workflows em condições de incerteza. *Anais do WCGA 2012 - X Workshop em Clouds, Grids e Aplicações*.
- Bittencourt, L. F., Madeira, E. R., and Da Fonseca, N. L. (2012). Scheduling in hybrid clouds. *Communications Magazine, IEEE*, 50(9):42–47.
- Bittencourt, L. F., Sakellariou, R., and Madeira, E. R. (2010). Dag scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm. In *Parallel, Distributed and Network-Based Processing (PDP), 18th Euromicro International Conference on 2010*, pages 27–34. IEEE.
- Cordeiro, D., Braghetto, K. R., Goldman, A., and Kon, F. (2013). Da ciência à e-ciência: paradigmas da descoberta do conhecimento. *Computação em nuvem*, (97):71–80A.
- Deelman, E., Gannon, D., Shields, M., and Taylor, I. (2009). Workflows and e-science: An overview of workflow system features and capabilities. *Future Generation Computer Systems*, 25(5):528–540.
- Guérout, T., Monteil, T., Da Costa, G., Neves Calheiros, R., Buyya, R., and Alexandru, M. (2013). Energy-aware simulation with DVFS. *Simulation Modelling Practice and Theory*, 39(0):76 – 91.
- Kliazovich, D., Bouvry, P., and Khan, S. U. (2012). GreenCloud: a packet-level simulator of energy-aware cloud computing data centers. *The Journal of Supercomputing*, 62(3):1263–1283.
- Rivoire, S., Ranganathan, P., and Kozyrakis, C. (2008). A comparison of high-level full-system power models. *HotPower*, 8:3–3.
- Topcuoglu, H., Hariri, S., and Wu, M.-y. (2002). Performance-effective and low-complexity task scheduling for heterogeneous computing. *Parallel and Distributed Systems, IEEE Transactions on*, 13(3):260–274.
- Vöckler, J.-S., Juve, G., Deelman, E., Rynge, M., and Berriman, B. (2011). Experiences using cloud computing for a scientific workflow application. In *Proceedings of the 2nd international workshop on Scientific cloud computing*, pages 15–24. ACM.
- Weiwei, C. and Deelman, E. (2012). WorkflowSim: A toolkit for simulating scientific workflows in distributed environments. In *IEEE 8th International Conference on E-Science (e-Science)*, pages 1–8.